

Correction du partiel

MU4IN501 – DLP : Développement d'un langage de programmation
Master STL, Sorbonne Université

Antoine Miné

Année 2020–2021

Cours 10
15 décembre 2020

Sujet : séquences

exemple

```
x = 1
y = {1*0: 2+2; 3+3; 4+4}
print(x+y)
=> 5
```

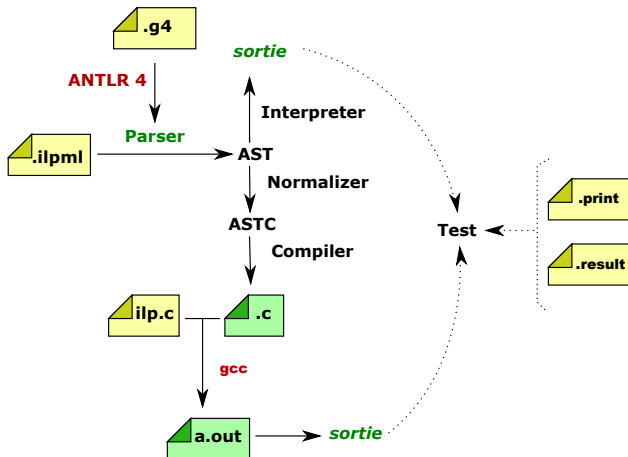
```
x = 0
y = {x+1: 2; 3; 4}
print(x+y)
=> 3
```

```
x = 1
y = {{x-1: 1}: 2; 3; 4}
print(x+y)
=> 4
```

- **Rappels** : définition d'une extension.
- **Questions 1–3** : Grammaire, AST, analyse syntaxique.
- **Question 4** : Interprète.
- **Question 5** : Compilateur, bibliothèque d'exécution.

Rappels

Rappels : structure d'ILP



Rappels : étapes d'une extension

Extension de la **syntaxe** :

- écrire une grammaire ANTLR 4 ;
- ajouter des nœuds **IAST**, **AST**, **ASTC** ;
- écrire un *Listener* obéissant à l'interface produite par ANTLR.

Extension de la **base de tests** (**.ilpml**, **.result**, **.print**).

Extension des **visiteurs** :

- classes **Interpreter**, **Normalizer**, **Compiler**.

Extension des **primitives** et **opérateurs** de l'interprète Java.

Extension de la **bibliothèque d'exécution C** :

- type **ILP_Object** dans **ilp.h** ;
- primitives dans **ilp.c**.

Extension des **classes de test** **InterpreterTest** et **CompilerTest**.

Ces étapes ne sont pas toutes nécessaires pour chaque extension.

Rappels : règles de programmation pour les extensions

En Java :

- **pas de modification du code existant** ;
- nouvelles classes dans des « packages » séparés
`com.paracamplus.ilp2.ilp2partiel...` ;
- réutilisation par **héritage** ;
- motifs **visiteur** et **composite** facilitant l'extensibilité.

En ANTLR 4 :

- difficile d'hériter d'une grammaire `.g4` pour y ajouter des règles ;
- si la grammaire change, la classe *Listener* ne peut pas être réutilisée ;
(ANTLR génère une nouvelle interface *Listener* sans lien d'héritage avec l'ancienne)

⇒ copie nécessaire, puis modification de la grammaire et du *Listener*.

En C :

- `ilp.c` et `ilp.h` implantent déjà tout ILP1 à ILP4...
- difficile d'étendre un type `struct`
⇒ autorisation de modifier `ilp.h` ;
- déclarer et définir les fonctions dans des `.c` et `.h` séparés.

Question 1–3 : Grammaire et AST

Grammaire ANTLR4

ILPMLgrammarPartiel.g4

```
grammar ILPMLgrammarPartiel;

@header { package antlr4; }

...
expr returns [com.paracampus.ilp1.interfaces.IASTexpression node]
// séquences de la grammaire précédente
: '(' exprs+=expr (','? exprs+=expr)* ','? ')' # Sequence

// séquences enrichies
| '{' index=expr ':' exprs+=expr (','? exprs+=expr)* ','? '}'
  # SequenceN
| ...
;

...
```

- ajout d'une règle dans `expr`
- attention à l'utilisation de `+=` et `*`

Interface d'AST

IASTsequenceN.java

```
package com.paracamplus.ilp2.ilp2partiel.interfaces;

import com.paracamplus.ilp1.interfaces.IASTexpression;

public interface IASTsequenceN extends IASTexpression
{
    IASTexpression[] getExpressions();
    IASTexpression  getIndex();
}
```

Reflet de la grammaire :

- une expression pour dénoter l'indice
- puis des expressions, comme pour un bloc

Interface de visiteur

IASTvisitor.java

```
package com.paracampus.ilp2.ilp2partiel.interfaces;

public interface IASTvisitor<Result, Data, Anomaly extends Throwable>
    extends com.paracampus.ilp2.interfaces.IASTvisitor<Result, Data, Anomaly>
{
    Result visit(IASTsequenceN iast, Data data) throws Anomaly;
}
```

- ajout du visiteur pour le nouveau nœud AST
- attention aux interfaces et classes définies avec le même nom mais dans des *packages* différents

Classe d'AST (1/2)

ASTsequenceN.java (début)

```
package com.paracamplus.ilp2.ilp2partiel.ast;
import com.paracamplus.ilp2.ilp2partiel.interfaces.IASTvisitor;
import ...

public class ASTsequenceN extends ASTexpression implements IASTsequenceN
{
    private IASTexpression[] expressions;
    private IASTexpression index;

    public ASTsequenceN(IASTexpression index, IASTexpression[] expressions) {
        this.expressions = expressions;
        this.index = index;
    }

    @Override public IASTexpression[] getExpressions()
    { return this.expressions; }

    @Override public IASTexpression getIndex()
    { return this.index; }
    ...
}
```

La classe est un simple conteneur avec :

- des attributs fixés par le constructeur
- et des getters

Classe d'AST (2/2)

ASTsequenceN.java (fin)

```
package com.paracampus.ilp2.ilp2partiel.ast;
import com.paracampus.ilp2.ilp2partiel.interfaces.IASTvisitor;
import ...

public class ASTsequenceN extends ASTexpression implements IASTsequenceN {

    ...

    @Override public <Result, Data, Anomaly extends Throwable>
    Result accept(com.paracampus.ilp1.interfaces.IASTvisitor<Result, Data, Anomaly>
        visitor, Data data) throws Anomaly
    {
        return ((IASTvisitor<Result, Data, Anomaly>) visitor).visit(this, data);
    }
}
```

- ne pas oublier de redéfinir **accept**
en utilisant la **bonne interface de visiteur** !

Fabrique d'AST

IASTfactory.java

```
package com.paracamplus.ilp2.ilp2partiel.interfaces;
...
public interface IASTfactory extends com.paracamplus.ilp2.interfaces.IASTfactory
{
    IASTexpression newSequenceN(IASTexpression index, IASTexpression[] asts);
}
```

ASTfactory.java

```
package com.paracamplus.ilp2.ilp2partiel.ast;
...
public class ASTfactory
extends com.paracamplus.ilp2.ast.ASTfactory implements IASTfactory
{
    @Override
    public IASTexpression newSequenceN(IASTexpression index, IASTexpression[] asts)
    {
        return new ASTsequenceN(index,asts);
    }
}
```

Ajout de la création de nœuds `ASTsequenceN`.

Listener ANTLR

ILPMLListener.java

```
package com.paracamplus.ilp2.ilp2partiel.parser;

import com.paracamplus.ilp2.ilp2partiel.interfaces.IASTfactory;
...
public class ILPMLListener implements ILPMLgrammarPartielListener
{
    protected IASTfactory factory;

    ...

    @Override
    public void enterSequenceN(SequencenContext ctx) { /* vide */ }

    @Override
    public void exitSequenceN(SequencenContext ctx) {
        ctx.node = factory.newSequenceN(ctx.index.node, toExpressions(ctx.exprs));
    }
}
```

- copie du *listener* ILP2
- utilisation de la nouvelle fabrique `factory` pour créer les nœuds

Lancement de l'analyse syntaxique ANTLR 4

ILPMLParser.java

```
package com.paracamplus.ilp2.ilp2partiel.parser;
import antlr4.ILPMLgrammarPartielLexer;
import antlr4.ILPMLgrammarPartielParser;
import com.paracamplus.ilp2.ilp2partiel.interfaces.IASTfactory;
...
public class ILPMLParser
extends com.paracamplus.ilp2.parser.ilpml.ILPMLParser
{
    public ILPMLParser(IASTfactory factory) { super(factory); }

    @Override public IASTprogram getProgram() throws ParseException
    {
        try {
            ANTLRInputStream in = new ANTLRInputStream(input.getText());
            ILPMLgrammarPartielLexer lexer = new ILPMLgrammarPartielLexer(in);
            CommonTokenStream tokens = new CommonTokenStream(lexer);
            ILPMLgrammarPartielParser parser = new ILPMLgrammarPartielParser(tokens);
            ILPMLgrammarPartielParser.ProgContext tree = parser.prog();
            ParseTreeWalker walker = new ParseTreeWalker();
            ILPMLListener extractor = new ILPMLListener((IASTfactory)factory);
            walker.walk(extractor, tree);
            return tree.node;
        } catch (Exception e) { throw new ParseException(e); }
    }
}
```

Question 4 : Interprète

Interprète (1/2)

Interpreter.java

```
package com.paracamplus.ilp2.ilp2partiel.interpreter;

import com.paracamplus.ilp2.ilp2partiel.interfaces.IASTvisitor;
...

public class Interpreter
extends com.paracamplus.ilp2.interpreter.Interpreter
implements IASTvisitor<Object, ILexicalEnvironment, EvaluationException>
{

    public Interpreter(IGlobalVariableEnvironment globalVariableEnvironment,
                      IOperatorEnvironment operatorEnvironment)
    {
        super(globalVariableEnvironment, operatorEnvironment);
    }

    ...
}
```

- implante notre nouvelle interface visiteur
- hérite de l'interprète d'ILP2
appel au constructeur du parent avec `super`

Interprète (2/2)

Interpreter.java

```

@Override public Object visit(IASTsequenceN iast, ILexicalEnvironment lexenv)
throws EvaluationException
{
    Object index = iast.getIndex().accept(this, lexenv);
    if ( !(index instanceof BigInteger) )
        throw new EvaluationException("The index must be an integer");
    int n = ((BigInteger ) index).intValue();

    IASTexpression[] expressions = iast.getExpressions();
    if (n < 0 || n >= expressions.length)
        throw new EvaluationException("Index out of range");

    Object ret = null;
    for (int i = 0; i < expressions.length; i++) {
        Object v = expressions[i].accept(this, lexenv);
        if (i == n) ret = v;
    }
    return ret;
}

```

- **évaluation** de l'index et **vérification** que c'est un entier **n**
- **vérification** que **n** est dans le bon intervalle
- **évaluation** de **toutes** les expressions suivant l'index
- retour de l'expression **ret**, calculée à l'indice **n** (**ret** ne peut pas rester NULL)

Question 5 : Compilateur

Classes et interfaces ajoutées (1/2)

ASTC :

- `IASTCsequenceN`
extends `IASTsequenceN`, `IASTCvisitable`
- `ASTCsequenceN`
extends `ASTsequenceN` implements `IASTCsequenceN`
- `INormalizationFactory`
extends
`com.paracamplus.ilp2.compiler.normalizer.INormalizationFactory`
- `NormalizationFactory`
extends
`com.paracamplus.ilp2.compiler.normalizer.NormalizationFactory`
implements `INormalizationFactory`

Classes et interfaces (2/2)

Normalisation :

- **Normalizer**

extends ILP2 Normalizer

implements IASTvisitor<IASTexpression,

INormalizationEnvironment, CompilationException>

Analyse statique :

- **GlobalVariableCollector**

extends ILP2 GlobalVariableCollector

implements IASTCvisitor<Set<IASTCglobalVariable>,

Set<IASTCglobalVariable>, CompilationException>

Génération de code :

- **Compiler**

extends ILP2 Compiler

implements IASTCvisitor<Void, Compiler.Context,

CompilationException>

Test :

- **CompilerTest**

extends ILP2 CompilerTest

Erreurs

Types d'erreurs :

- l'index s'évalue en une valeur non entière
- l'index s'évalue en une valeur négative ou trop grande

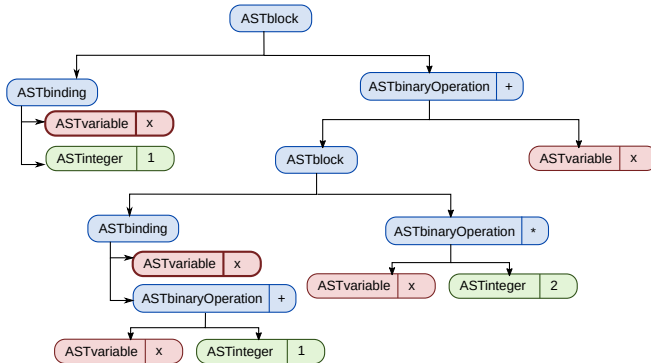
⇒ erreurs **dynamiques** (pas d'erreur statique)

Détectables uniquement à l'exécution, pas à la compilation.

Nous pouvons réutiliser `ILP_domain_error(char*, ILP_Object)` par exemple pour signaler l'erreur dans le code C généré.

Rappels : ASTC normalisé (1/4)

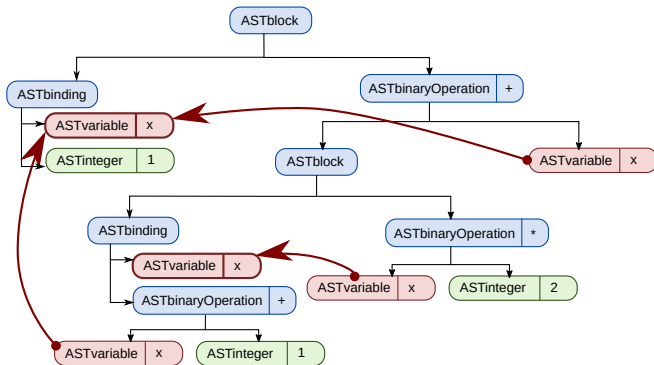
```
let x = 1 in
  (let x = x + 1 in 2 * x) + x
```



Problème : définition et utilisation de variables de même nom (x).

Rappels : ASTC normalisé (2/4)

```
let x = 1 in
  (let x = x + 1 in 2 * x) + x
```



Solution : **lier** chaque utilisation d'une variable à sa définition.

Rappels : ASTC normalisé (3/4)

Le compilateur commence par transformer l'AST en ASTC.

Classification :

- distinction de type d'objet C et de portée :
 - fonction globale : `IASTCglobalFunctionVariable` ;
 ⇒ correspond à une fonction C
 - variable locale : `ASTClocalVariable` ;
 - variable globale : `ASTCglobalVariable` ;
 ⇒ correspond à une variable C contenant un `ILP_Object`

Il faut distinguer une fonction d'une variable globale contenant une fonction
e.g., dans `let f = double`, `f` est une `IASTCglobalVariable`
`double` est une `IASTCglobalFunctionVariable`

Une variable référence une fonction avec une valeur de type `ILP_Closure`.

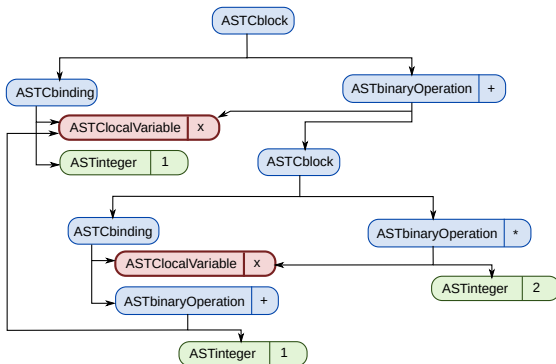
- distinction du type d'appel :
 - direct, par nom : `ASTCglobalInvocation`;
 - indirect, par variable : `ASTCcomputedInvocation`.

Partage et identification :

- une **ASTCvariable** identifie de manière unique un identificateur ILP et C ;
- toutes les utilisations de la même variable partagent le même nœud.

Rappels : ASTC normalisé (4/4)

```
let x = 1 in
  (let x = x + 1 in 2 * x) + x
```



Normalisation de `sequenceN`

Normalizer.java

```
package com.paracampus.ilp2.ilp2partiel.compiler.normalizer;
...
public class Normalizer extends com.paracampus.ilp2.compiler.normalizer.Normalizer
{
    @Override
    public IASTExpression visit(IASTsequenceN iast, INormalizationEnvironment env)
    throws CompilationException
    {
        IASTExpression index = iast.getIndex();
        IASTExpression newIndex = index.accept(this, env);

        IASTExpression[] expressions = iast.getExpressions();
        IASTExpression[] exprs = new IASTExpression[expressions.length];
        for ( int i=0 ; i< expressions.length ; i++ )
            exprs[i] = expressions[i].accept(this, env);

        return getFactory().newSequenceN(newIndex, exprs);
    }
}
```

- normalisation récursive
- construction d'un ASTC à partir de l'AST

Rappels : collecte des variables locales et globales

Le compilateur utilise plusieurs passes de visiteurs sur l'ASTC :

- **GlobalVariableCollector**

Collecte les **variables et fonctions globales**.

La liste de toutes les globales ILP doit être connue **avant** de générer le code C, pour générer les **déclarations et prototypes C** correspondant aux objets globaux.

- **FreeVariableCollector**

Collecte les **variables libres**, nécessaire pour les clôtures (ILP3).

⇒ nous ne l'implantons pas ici (ILP2)...

Collecte des variables globales dans `sequenceN`

GlobalVariableCollector.java (partiel)

```
public class GlobalVariableCollector
    extends com.paracampus.ilp2.compiler.GlobalVariableCollector
    implements IASTCvisitor< Set<IASTCglobalVariable>,
                             Set<IASTCglobalVariable>,
                             CompilationException >
{
    @Override
    public Set<IASTCglobalVariable>
    visit(IASTsequenceN iast, Set<IASTCglobalVariable> data)
        throws CompilationException
    {
        result = iast.getIndex().accept(this, result);
        for ( IASTexpression exp : iast.getExpressions() )
            result = exp.accept(this, result);
        return result;
    }
}
```

- parcours récursif pour accumuler les variables globales
- retourne l'ensemble `data` passé en argument, enrichi des variables globales du sous-arbre

Rappels : motivation pour le schéma de compilation

Problème : Comment compiler `(let x = 2 in x + 1) * 2`?

difficulté : en C classique un bloc ne peut pas retourner de valeur

La classe `Compiler`, en Java, génère du C :

- par parcours récursif de l'ASTC
e.g. : évaluer les arguments d'un opérateur, avant d'évaluer l'opérateur
- en utilisant des variables temporaires
e.g. : stocker le résultat de la compilation d'une expression
- qui fournit le résultat de l'évaluation au code englobant
en ILP, tout est expression, tout renvoie une valeur

Le schéma de compilation $\xrightarrow{d}$ présente ces étapes de manière concise :

- en donnant le code C généré plutôt que le code Java qui le génère
- en restant générique grâce à un « code à trou » (appels récursifs)
- en utilisant un contexte `d` pour savoir que faire de la valeur de retour

Rappel : schéma de compilation de l'addition

Schéma de compilation

```

       $\xrightarrow{d}$ 
     $arg1 + arg2$ 
    -----
    {
      ILP_Object tmp1;
      ILP_Object tmp2;
       $\xrightarrow{(tmp1=)}$ 
       $arg1$ 
       $\xrightarrow{(tmp2=)}$ 
       $arg2$ 
       $d$  ILP_Plus(tmp1, tmp2);
    }
```

- création de **temporaires** `tmp1`, `tmp2`, délimitées par un bloc C `{ ... }`
- calcul des arguments par appel récursif au générateur de code avec **redirection dans les temporaires**
ainsi, le code généré pour `arg1` et `arg2` peut être des instructions C qu'il serait impossible d'utiliser en argument de `ILP_Plus`
- calcul final de l'addition dans la destination `d` originale avec `ILP_Plus`

Rappel : schéma de compilation de la séquence

Schéma de compilation

```

       $\longrightarrow^d$ 
expr1; ...; exprN - 1; exprN
-----
{
    ILP_Object tmp;
     $\longrightarrow^{(tmp=)}$ 
    expr1 ;

    ...

     $\longrightarrow^{(tmp=)}$ 
    exprN - 1 ;
     $\longrightarrow^{(tmp=)}$ 
    exprN ;
    d tmp
}

```

- les expressions *expr1* à *exprN* - 1 sont évaluées dans un temporaire *tmp* et leur valeur est **ignorée** ! (on aurait pu aussi utiliser $\longrightarrow^{(void)}$ *expr*)
- la dernière expression est évaluée dans *tmp* qui est **appliquée à la destination originale**

Schéma de compilation de sequenceN

Schéma de compilation

$$\frac{\text{index} : \text{expr1}; \dots; \text{exprN}}{\{$$

```

    ILP_Object idx, tmp, ret;
    →(idx=)
    index ;
    if (!ILP_isInteger(idx)) ILP_domain_error(...)
    int n = idx->_content.asInteger;
    if (n < 0 || n >= N - 1) ILP_domain_error(...)
    →(tmp=)
    expr1 ; if (n == 0) ret = tmp;
    ...
    →(tmp=)
    exprN ; if (n == N - 1) ret = tmp;
    d ret;
}
```